

北京大学信息科学技术学院

程序设计实习课程大作业

学海漫游：异世界的信科少女

Study Adventure: The Informatics Girl in Another World

组 号： yyy

成员 A（剧情核心）： 代易瓚

成员 B（界面核心）： 闫瑾

成员 C（玩法核心）： 沈昱萱

提交日期： 2026 年 7 月 6 日

代码仓库： <https://github.com/look-dyz/StudyAdventure>

Contents

1	程序功能介绍	2
1.1	项目背景与总体定位	2
1.2	核心玩法循环	2
1.3	五个知识小游戏	2
1.4	多维数值系统	3
1.5	五条结局路线	3
1.6	精美的主界面与人物立绘	3
1.7	跨平台支持	3
2	项目各模块与类设计细节	3
2.1	总体架构	3
2.2	公共基类模块 (common)	4
2.3	核心系统模块 (core, 成员 C 负责)	4
2.4	剧情系统模块 (story, 成员 A 负责)	4
2.5	界面与地图模块 (ui / map, 成员 B 负责)	5
2.6	小游戏模块 (games, 成员 C 负责)	5
2.7	关键设计模式与工程实践	6
3	小组成员分工情况	6
3.1	分工原则	6
3.2	协作流程	6
4	项目总结与反思	6
4.1	已完成的工作	6
4.2	存在的不足	7
4.3	收获与反思	7

1 程序功能介绍

1.1 项目背景与总体定位

《学海漫游：异世界的信科少女》是一款使用 C++17 与 Qt 6 (Widgets + Multimedia) 开发的单机养成模拟游戏，作为北京大学信息科学技术学院程序设计实习课程的三人合作大作业。项目将“恋爱养成游戏”的叙事框架与“课程知识小游戏”相结合：玩家扮演一名信科大一新生，在一次期末复习的深夜穿越到异世界，与四门专业课程拟人化而成的角色——程序设计、高等数学、线性代数、人工智能引论——展开为期五周的校园生活，并在剧情推进过程中通过完成与学科相关的知识小游戏来提升好感度，最终走向五种不同结局之一。

1.2 核心玩法循环

游戏的主循环由“地图探索 → 事件触发 → 剧情对话 → 小游戏挑战 → 数值结算 → 周日推进”构成：

- 1. 地图探索：玩家在教学楼、图书馆、宿舍、未名湖等四张可点击地图场景间移动，选择当天的行动地点；
- 2. 事件触发：进入图书馆跳转至小游戏；未名湖与宿舍用于调节压力值与推进日程；
- 3. 剧情对话：对话框呈现角色立绘与台词，玩家在关键节点做出选择；
- 4. 小游戏挑战：部分事件会跳转至与该学科相关的知识小游戏，玩家的表现将直接影响数值反馈；
- 5. 数值结算：小游戏结果与剧情选择共同驱动好感度、压力值、黑化值等数值变化；
- 6. 周日推进：五周时间线走完后，系统依据数值状态判定并展示最终结局。

1.3 五个知识小游戏

项目最具特色的部分是将课程知识点包装为可玩的小游戏，每个小游戏对应一门课程的核心算法或知识点。经小组讨论，最终版本聚焦于五个小游戏，均已实现完整可玩流程；原计划的线性代数“矩阵运算”小游戏因课程时间限制，最终版本中未纳入实现范围：

小游戏	对应学科	核心算法 / 知识点
扫雷 (Minesweeper)	程序设计	二维数组遍历、递归展开
21 点 (Blackjack)	高等数学	随机数、状态机、可完整游玩
井字棋 (Tic-Tac-Toe)	程序设计	Minimax + α - β 剪枝
AI 迷宫 (AI Maze)	人工智能引论	BFS 最短路径、递归回溯生成迷宫
记忆翻牌 (Memory Match)	线性代数	卡牌随机洗牌、状态匹配判定

Table 1: 五个知识小游戏一览

五个小游戏（扫雷、21 点、井字棋、AI 迷宫、记忆翻牌）均已实现完整可玩流程，恰好覆盖程序设计、高等数学、人工智能引论、线性代数四门学科，每门学科均对应至少一个小游戏，好感度提升渠道更加均衡。

1.4 多维数值系统

玩家的成长状态由 **Player** 类统一维护，包括：

- 四门学科的好感度（程设 / 高数 / 线代 / AI 引论各一项）；
- 全局压力值，随剧情推进与小游戏表现波动，影响事件走向；
- 线性代数专属的黑化值，用于解锁“阴郁占有”角色线的特殊结局分支；
- 当前游戏日期（第几周 / 第几天），驱动时间线与剧本调度。

数值变化通过 Qt 信号槽机制实时反映在界面顶部的状态栏（**StatusBar**）中，好感度以进度条形式动态刷新，为玩家提供即时反馈。

1.5 五条结局路线

依据四条学科好感线与黑化值的综合判定，游戏设置了五种结局，由 **EndingJudge** 按照预设的优先级链进行判定，保证多个结局条件同时满足时也能得到确定性结果。五周完整剧情与全部五条结局剧本均已编写完成，玩家可从开局一路游玩至任一结局，形成完整闭环体验。

1.6 精美的主界面与人物立绘

项目在美术呈现上投入了较多打磨：设计了风格统一、视觉美观的游戏主界面（主菜单、地图场景、对话界面），并为四个学科拟人化角色分别绘制了专属人物立绘，在对话与好感度提升等关键节点展示，增强了角色辨识度与代入感，使整体呈现效果超越了课程作业的一般水准。

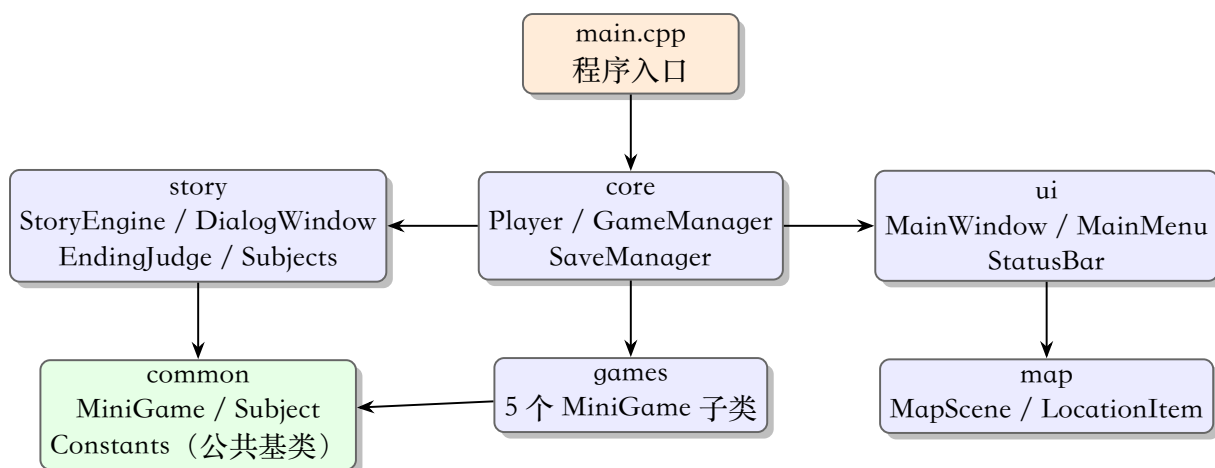
1.7 跨平台支持

项目基于 CMake (≥ 3.16) 与 Qt 6 构建，通过 GitHub Actions 持续集成在 Windows、macOS、Linux 三平台上自动验证编译，实现“一份代码，三端运行”。

2 项目各模块与类设计细节

2.1 总体架构

项目采用分层、职责分离的模块化架构，源代码位于 **src/** 目录下，共划分为五大功能域：



GameManager 作为全局唯一的调度中枢，持有 **Player** 实例并负责场景切换与小游戏结果的统一奖惩处理；**common** 目录下的 **Subject** 与 **MiniGame** 是全项目复用的抽象基类，由三名成员共同遵守其接口约定进行并行开发。

2.2 公共基类模块（common）

- **Subject**：学科抽象基类，定义学科名称、好感度访问接口与 `createGame()` 工厂方法，由 `ProgDesignSubject`、`CalculusSubject`、`LinearAlgebraSubject`、`AIIntroSubject` 四个派生类实现，体现工厂模式——通过基类指针即可创建对应学科的小游戏实例。
- **MiniGame**：所有小游戏的抽象基类，统一定义 `start()` 启动接口与 `finished` 信号，使 **GameManager** 可以用统一的方式管理五个风格迥异的小游戏。
- **Constants**：存放好感度阈值、压力值上下限等游戏平衡性相关的全局常量，避免魔数散落在各模块中。

2.3 核心系统模块（core，成员 C 负责）

- **Player**：数值系统核心，封装四项学科好感度、压力值、线代黑化值与当前日期，并通过 `affinityChanged` 等 Qt 信号在数值变化时通知界面层，体现封装与信号槽解耦。
- **GameManager**：以单例模式实现（`GameManager::instance()`），是连接剧情、界面、小游戏三大模块的调度中心，负责场景切换逻辑，并统一处理各小游戏 `finished` 信号所带来的数值奖惩。
- **SaveManager**：负责存档/读档，将 **Player** 与 **StoryEngine** 的状态序列化到本地文件，保证游戏进度可持久化。

2.4 剧情系统模块（story，成员 A 负责）

- **StoryEngine**：剧情驱动核心，负责加载 JSON 剧本、解析分支跳转条件与 `effects`（数值影响）字段，并将执行结果应用到 **Player**；
- **DialogWindow**：对话框组件，负责呈现角色立绘与台词，支持逐字显现效果与玩家分支选择；

- **EndingJudge**: 结局判定器，按预设优先级链依次检查五种结局的达成条件，保证判定结果的确定性与可扩展性；
- **Subjects**: 四个学科角色的具体人设数据与专属剧情钩子。

数据驱动设计是本模块的核心亮点：剧情内容全部存储在 `assets/scripts/` 目录下的 JSON 文件（如 `week1.json`、`route_progdesign.json`）中，与 C++ 代码完全解耦，编剧无需掌握 C++ 即可编写/修改剧情，也无需重新编译程序。最终版本中，五周完整剧情与全部五条结局剧本均已编写完成并接入 StoryEngine。

2.5 界面与地图模块 (ui / map, 成员 B 负责)

- **MainWindow**: 主窗口框架，管理五个场景页面之间的切换 (QStackedWidget 风格)；
- **MainMenu**: 主菜单组件，提供开始游戏、读取存档、退出等入口；
- **StatusBar**: 顶部状态栏，通过连接 `Player::affinityChanged` 信号自动刷新好感度进度条，无需手动轮询；
- **MapScene / LocationItem**: 基于 QGraphicsScene 的可交互地图，玩家可点击教学楼、图书馆、宿舍、未名湖四个地点触发对应事件。

在美术呈现方面，成员 B 设计并完成了风格统一、视觉美观的游戏主界面，并为四个学科拟人化角色分别绘制了专属人物立绘，在对话框与好感度变化等关键节点展示，显著提升了游戏的观感与代入感。

2.6 小游戏模块 (games, 成员 C 负责)

五个小游戏均继承自 MiniGame 基类，各自封装独立的算法逻辑，均已实现完整可玩流程，并按下表与四门学科一一对应：

- **MinesweeperGame** (程序设计)：二维网格数据结构 + 递归展开空白区域，含完整的插旗、计数与胜负判定逻辑；
- **TicTacToeGame** (程序设计)：内置 Minimax + α - β 剪枝算法实现 AI 对手，已实现完整可玩流程；
- **BlackjackGame** (高等数学)：牌堆随机洗牌、点数结算状态机，已实现完整可玩流程；
- **AIMazeGame** (人工智能引论)：递归回溯法生成随机迷宫，BFS 求解最短路径用于寻路提示，已实现完整可玩流程；
- **MemoryMatchGame** (线性代数)：线代概念与性质配对的记忆翻牌游戏，卡牌内容由 JSON 术语库驱动，核心逻辑为洗牌算法、翻牌状态机与配对判定，已实现完整可玩流程。

原计划的“矩阵运算 (Matrix Game)”因课程时间限制，最终版本中决定不予实现；线性代数学科线的好感度反馈由 MemoryMatchGame (记忆翻牌) 与剧情选择共同驱动。

2.7 关键设计模式与工程实践

Listing 1: 工厂模式：由学科对象创建对应的小游戏实例

```
1 Subject* progdesign = new ProgDesignSubject;  
2 MiniGame* game = progdesign->createGame(); // 工厂模式  
3 game->start();
```

Listing 2: 单例模式与信号槽解耦

```
1 auto& mgr = GameManager::instance(); // 单例模式  
2 mgr.player()->addAffinity(SubjectType::Calculus, +5);  
3  
4 // Player 与 StatusBar 解耦  
5 connect(player, &Player::affinityChanged,  
6         statusBar, &StatusBar::onAffinityChanged);  
7  
8 // MiniGame 与 GameManager 解耦  
9 connect(game, &MiniGame::finished,  
10         &GameManager::instance(), &GameManager::onMiniGameFinished);
```

除面向对象设计外，项目在工程规范上也做了较为完整的建设：使用 CMakeLists.txt 统一管理跨平台构建（启用 AUTOMOC/ AUTORCC/ AUTOUIIC）；通过 GitHub Actions 在三平台上持续集成验证编译；配合 .clang-format、.editorconfig 统一代码风格；并以 CODEOWNERS 机制自动分配 Pull Request 的代码评审人，保障三人协作的代码质量。

3 小组成员分工情况

3.1 分工原则

项目按照“剧情 / 界面 / 玩法”三大功能域，将开发任务拆分给三名成员并行推进，并在开发初期约定统一的类接口（详见仓库 docs/□ □ □ □ .md），以降低模块间联调时的耦合成本。

3.2 协作流程

项目按照六个阶段推进：设计与接口约定（5/13 – 5/15）、搭骨架并于 5/17 跑通最小可运行示例、正式并行开发（5/18 – 5/28）、联调与测试（5/28 – 5/31）、撰写作业报告（5/31 – 6/02）、录屏与最终交付（6/02 – 6/06）。三人通过统一的 CODEOWNERS 规则进行代码评审，并使用 Issue / Pull Request 模板规范协作沟通，日常协作细节记录于 docs/□ □ □ □ .md。

4 项目总结与反思

4.1 已完成的工作

- 完成了五大模块（剧情 / 界面 / 地图 / 玩法 / 数值）的完整开发与联调；

成员	负责模块	主要工作内容
成员 A	剧情核心	StoryEngine 剧情引擎（分支跳转、条件判断、effects 应用）、 DialogWindow 对话框组件、 EndingJudge 结局判定链、五周完整 JSON 剧本内容编写与四条学科线、五条结局的设计
成员 B	界面核心	MainWindow 主框架与五个场景页面管理、 MainMenu 主菜单、 StatusBar 状态栏（信号槽驱动）、 MapScene / LocationItem 可交互地图、美观主界面视觉设计与四位角色专属人物立绘绘制
成员 C	玩法核心	五个小游戏（扫雷、21 点、井字棋、AI 迷宫、记忆翻牌）的完整算法实现、 Player 数值系统、 GameManager 场景与奖惩调度、 SaveManager 存档系统

Table 2: 三人分工一览

- 五个知识小游戏（扫雷、井字棋含 Minimax、21 点、AI 迷宫含 BFS 与递归回溯、记忆翻牌）均已实现完整可玩流程，一一对应程序设计、高等数学、人工智能引论、线性代数四门学科；
- 五周完整剧情与全部五条结局剧本均已编写完成，玩家可从开局完整游玩至任一结局；
- 设计并完成了风格统一、美观精致的游戏主界面，并为四个学科角色绘制了专属人物立绘；
- 建立了数据驱动的 JSON 剧本体系，实现了策划与工程的解耦；
- 搭建了较为完善的工程化基础设施：跨平台 CMake 构建、三平台 CI、代码风格统一、PR/Issue 模板与自动化代码评审分配。

4.2 存在的不足

- 经小组讨论，最终版本未实现原计划的“矩阵运算”小游戏，线性代数学科线目前仅由记忆翻牌与剧情选择共同驱动，运算类的实操玩法维度略显单薄；
- 音效与背景音乐资源尚未完整接入 **Multimedia** 模块；
- 存档系统的异常处理（如文件损坏、版本不兼容）覆盖尚不全面；
- 由于开发周期集中于剧情与美术打磨，小游戏与结局的整体数值平衡性测试还不够充分，个别数值阈值可能需要后续微调。

4.3 收获与反思

通过本次大作业，小组成员在以下几方面获得了实际锻炼：

- 面向对象设计能力：在 **Subject/MiniGame** 等公共基类的设计过程中，切身体会到抽象接口先行、实现细节后置的好处——正是有了清晰的基类约定，三人能够在几乎不冲突的情况下并行开发六个风格迥异的小游戏与四个学科分支；

- **设计模式的实践价值：**单例模式解决了全局状态（GameManager）的一致性问题，工厂模式让“学科 → 小游戏”的映射关系一目了然，信号槽机制则从根本上避免了界面层对数值层的轮询式查询，让代码更加松耦合、易维护；
- **数据驱动思想：**将剧情内容从代码中剥离为 JSON 文件，是本项目在工程实践上最大的收获之一，它不仅降低了非程序成员参与内容创作的门槛，也让后期修改剧情不再需要重新编译整个项目；
- **团队协作与工程规范：**通过提前约定接口、使用 CI 持续验证编译、统一代码风格，小组在“先设计后编码”上体会到了明显优于“边做边改”的协作效率，也认识到接口一旦约定不清晰将给后期联调带来较大的返工成本；
- **取舍与时间管理：**受限于课程时间，小组在开发中期主动对功能范围做了取舍——放弃了优先级相对较低的“矩阵运算”小游戏，将更多精力投入到五周完整剧情、五条结局以及主界面与人物立绘的美术打磨上。事后来看来，这一取舍使项目最终呈现出的完整度与观感明显优于面面俱到但处处半成品的方案，也让我们体会到在有限时间内明确核心优先级、敢于砍掉次要功能的重要性。

后续如有时间，小组计划补充“矩阵运算”小游戏以进一步丰富线性代数学科线的玩法多样性，并进一步接入完整的音效与背景音乐、加强存档系统的健壮性与整体数值平衡性测试。